

Contents

Guide 7 — Web Data Aggregation, the simple version	1
1. The idea, in three sentences	1
2. Three ways to get the data — and why we scrape	1
3. The pipeline (the same five steps for every source)	1
4. Normalising to a common schema (the heart of aggregation)	1
5. Where each part comes from (the research)	2
6. A worked example	2
7. The files	2
8. Jury questions — ready answers	3

Guide 7 — Web Data Aggregation, the simple version

How Mobile Match Algeria turns three separate operator websites into **one** clean, comparable catalogue — in plain words, with the research, a worked example, and ready answers for the jury. (This is the big picture; Guide 1 covers the scraping mechanics.)

1. The idea, in three sentences

The plans live on three different websites (Djezzy, Ooredoo, Mobilis), each in its own format and even its own language. Aggregation is the job of **collecting** all of them, **reshaping** them into one common format, and storing them in a **single catalogue** the app can search and compare. Scraping is only the *fetching* step; aggregation is the whole “three messy sources in, one clean table out” pipeline.

2. Three ways to get the data — and why we scrape

The literature lists three ways to collect web data, in order of preference:

1. **A public API** — best when the data owner offers a documented endpoint. Fast and already structured.
2. **A syndication feed (RSS/Atom)** — only exists for blogs/news, not for operator plans.
3. **Web scraping** — the fallback: read the public web page directly. No cooperation from the operator needed.

The three Algerian operators **do not publish an API** for their plans, but the prices are openly visible to any browser — so scraping is the established, defensible choice here.

3. The pipeline (the same five steps for every source)

Online source → Fetch → Parse & extract → Normalise & validate → Database → App

1. **Fetch** the operator’s plan page (a real headless browser — see Guide 1).
2. **Parse & extract** the raw values from the HTML (price, data, validity, ...).
3. **Normalise** every value into one common format (the hard part — see §4).
4. **Validate** each record against a set of sanity rules; bad records are dropped.
5. **Store** into one `offer` table and serve it to the app.

The point: after this, the app never deals with three formats — it queries **one** table.

4. Normalising to a common schema (the heart of aggregation)

Each operator writes things differently, so before storing we convert everything to the same shape:

- **Units** → gigabytes: “500 Mo” becomes 0.5, “15 Go” becomes 15.
- **Prices** → a plain number in DA: “1 000 DA” becomes 1000.
- **Validity** → days: “1 mois” becomes 30.
- **Unlimited / none** → sentinels: unlimited calls or SMS is stored as -1, none as 0, so the app can tell “unlimited” apart from “zero”.
- **Language / noise** → feature text is cleaned and trimmed of promotional suffixes.
- **Duplicates** → each plan gets a `slug` and is keyed by (*operator*, *slug*), so the same plan scraped twice (carousels often duplicate cards) is stored only once.

Every record then passes a **validation gate** (a set of sanity rules — price and data in plausible ranges, data ≠ price, etc.); records that fail are discarded rather than stored.

5. Where each part comes from (the research)

Part	What it does	Comes from
API > feed > scraping choice	pick how to collect	the web-data-collection literature — scraping is the standard fallback when no API exists: Boegershausen & Datta (2022) (scraping ≈ 59 % of reviewed studies vs ≈ 12 % for APIs); Glez-Peña et al. (2014) , “ <i>Web scraping technologies in an API world</i> ”
Fetch → parse → normalise → validate → store	turn many sources into one clean table	the standard aggregation pipeline (extract, transform, load)

What is *ours* is the configuration: the common `offer` schema, the unit conversions and the -1/0 sentinels, the (*operator*, *slug*) deduplication key, the validation rules, and the daily scheduled run.

6. A worked example

On Djazzy’s page a plan shows “**15 Go**”, “**1 000 DA**”, “**1 mois**”, unlimited Djazzy calls. Aggregation turns that into one normalised record:

```
{ operator: "Djazzy", name: "Djazzy LEGEND 1000", type: PREPAID,
  dataGB: 15, priceDA: 1000, validityDays: 30,
  voiceMinutes: -1, smsCount: -1, network: "4G" }
```

The same five steps run for Ooredoo and Mobilis, so a “15 GB” Djazzy plan and a “15 GB” Mobilis plan end up in the **same table with the same field names** — which is exactly what lets the search, filters, comparison, and recommendation engine treat all three operators identically. If a stored plan’s price changed since the

last run, a PriceHistory row is added and a price-drop alert is sent; plans that vanished from a page are marked inactive.

7. The files

- [scraper/fetch.ts](#) — the fetch step (headless browser, retries).
 - [scraper/djezzy.ts](#), [ooredoo.ts](#), [mobilis.ts](#) — parse & extract, per operator.
 - [scraper/validate.ts](#) — normalise (parseGB, parsePriceDA, parseValidityDays, cleanFeatureText) and validateOffer.
 - [scraper/index.ts](#) — deduplicate, upsert into the one offer table, track price history, deactivate stale plans, log the run.
-

8. Jury questions — ready answers

- **Why not just use the operators' APIs?** They don't publish one for their plans. The data is public in the browser but not as an API, which is the exact case where scraping is the accepted method (Boegershausen & Datta 2022; Glez-Peña 2014).
 - **What is the difference between scraping and aggregation?** Scraping is one step (fetching a page). Aggregation is the whole pipeline that turns three different sources into one clean, comparable catalogue — fetch, normalise, validate, store.
 - **Why normalise at all?** Because the operators use different units, languages, and wordings. Converting everything to one schema (GB, DA, days, sentinels) is what makes the three operators comparable in a single query.
 - **What stops duplicates?** Each plan is keyed by (*operator*, *slug*), so the same plan seen twice in one scrape (carousels clone cards) is stored once.
 - **How do you keep it fresh?** A scheduled daily run repeats the pipeline, adds new plans, records price changes, and deactivates plans that disappeared.
-

Last core concept guide. The remaining one if you want it: responsive design & accessibility.